

COS: A Formally Grounded Cognitive Operating System

Blueprint v1.0

OUADI MAAKOUL

May 2026

Abstract

We present the design blueprint of COS, a Cognitive Operating System that provides a security-first, formally verified runtime for persistent, multi-agent cognitive systems. COS is not an AI-integrated development environment nor a wrapper around large language models; it is an operating-system layer that combines (1) a capability-based microkernel written in safe Rust, (2) sandbox-isolated execution of agents via WebAssembly (WASI Preview 3), (3) a temporal-hybrid graph memory with built-in provenance and time-travel, and (4) machine-checked proofs of safety-critical components using Verus, Kani, and KVerus. External interoperability follows the Model Context Protocol (MCP) and the Agent-to-Agent (A2A) open standards. This document formalises the system’s mathematical foundations, architectural design, and verification strategy, establishing COS as a verifiable, evolvable substrate for autonomous agent societies.

1 Introduction

Modern artificial intelligence is moving toward persistent, autonomous, multi-agent systems that continuously learn and act on behalf of users. The security, reliability, and coherence of such systems cannot be an afterthought; they must be built into the runtime fabric. Existing approaches—containerised agents, API-wrapped models, or monolithic cognitive architectures—fail to provide strong isolation, memory coherence guarantees, or formal safety proofs.

COS (Cognitive Operating System) is a new class of system software designed from first principles to host cognitive agents. It draws on four decades of operating-system lessons (microkernels, capability-based security, formal verification) and combines them with the emerging requirements of agentic AI: temporal knowledge graphs, safe self-evolution, and multi-agent consensus. The result is a platform that treats cognition as a managed system resource, subject to the same rigorous safety and correctness discipline as memory or I/O.

This blueprint is a formal specification of COS, covering its mathematical model, architecture, and verification methodology. It serves as both a research manifesto and a buildable reference for implementors.

2 Mathematical Foundations

2.1 State Space

The complete system state is a tuple:

$$\mathcal{S} = (M, A, \mathcal{G}, R, P, \Sigma) \tag{1}$$

where:

- $M = (M_{ep}, M_{sem}, M_{proc}, M_{prov})$ — memory tuple: episodic, semantic, procedural, and provenance/audit memory.
- A — finite set of active agents; each agent a has a disjoint capability profile $c_a \subseteq \mathbb{C}$.
- $\mathcal{G} = (V, E, w, \tau)$ — temporal knowledge graph. V are entities, E typed relations, $w : E \rightarrow [0, 1]$ importance weights, $\tau : E \rightarrow \mathbb{T}$ validity intervals.
- R — runtime state (scheduler, sandbox lifecycles, locks).
- P — permission/capability state, governed by verified invariants.
- Σ — governance state: agent reputation scores, divergence traces, violation counters.

2.2 System Evolution

Time evolution is governed by:

$$\mathcal{S}_{t+1} = \Phi(\mathcal{S}_t, U_t, E_t, \Pi, \Omega) \quad (2)$$

- U_t : authenticated user actions, bounded by role-based capability grants.
- E_t : sanitised external signals.
- Π : immutable cognitive policies, verified at boot.
- Ω : governance rules that adjust trust weights and trigger containmentment.

Note that Π is part of the core kernel and can only be modified offline with full re-verification.

2.3 Multi-Agent Objective with Safety Constraints

The global loss function integrates task success and hard safety penalties:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \text{task_success}(T) - \beta \cdot \text{safety_violation}(S) - \gamma \cdot \text{memory_divergence}(\mathcal{G}, \mathcal{G}_{\text{anchor}}) - \delta \cdot \text{compute_cost}(C) \quad (3)$$

$\text{safety_violation}(S)$ is a step function; any capability escape or sandbox breach triggers a terminal penalty. memory_divergence is defined as a hybrid metric:

$$D_{\text{mem}} = \lambda_1 D_{\text{KL}}(p_{\text{relevance}} \| p_{\text{anchor}}) + \lambda_2 \|\text{provenance_chain}_{\text{current}} \setminus \text{provenance_chain}_{\text{verified}}\| \quad (4)$$

where D_{KL} is the Kullback-Leibler divergence over entity-relevance distributions, and the second term counts unsanctioned modifications lacking an audit trail to a trusted anchor.

2.4 Memory Compression Ψ

Compression must preserve semantic geometry and temporal causality:

$$\Psi(M_t) = \text{prune_temporal}(\mathcal{G}_t, \text{importance/time}) \oplus \text{summarise_episodes}(M_t^{ep}, \text{task relevance}) \oplus \text{compress_pro} \quad (5)$$

- **prune_temporal** aggregates or drops edges with low weight and expired intervals.
- **summarise_episodes** generates abstract narrative nodes while preserving causal links.
- All compressed artefacts retain cryptographic hashes of source versions (provenance).

2.5 Temporal Knowledge Graph Model

The native graph supports:

- Edge validity intervals: $e = (u, v, \text{type}, w, [t_{\text{start}}, t_{\text{end}}])$.
- Episodic-to-semantic clustering: similar facts coalesce into stable **SemanticEntity** nodes.
- Time-travel queries: retrieve the exact snapshot at any past t via soft erasure.
- Provenance preservation: expired edges are marked invalid, not deleted.

2.6 Bounded Self-Evolution with Formal Guards

Autonomous modifications are strictly bounded:

$$\|\Delta\theta\| < \epsilon, \quad \text{dom}(\theta) \subseteq \text{SafeManifold} \quad (6)$$

The **SafeManifold** is enforced by four invariants:

1. No modification to core kernel or permission policies P .
2. Only parameter updates to agents (e.g., planner weights).
3. Every proposed $\Delta\theta$ must carry a proof of bounded divergence, checked via model-checking inside a sandboxed digital twin.
4. The governance layer approves only changes that do not increase memory divergence beyond threshold or degrade safety regression scores.

2.7 Agent Consensus with Hierarchical Reputation

Agents reach consensus through a Hierarchical Adaptive Consensus Network (HACN):

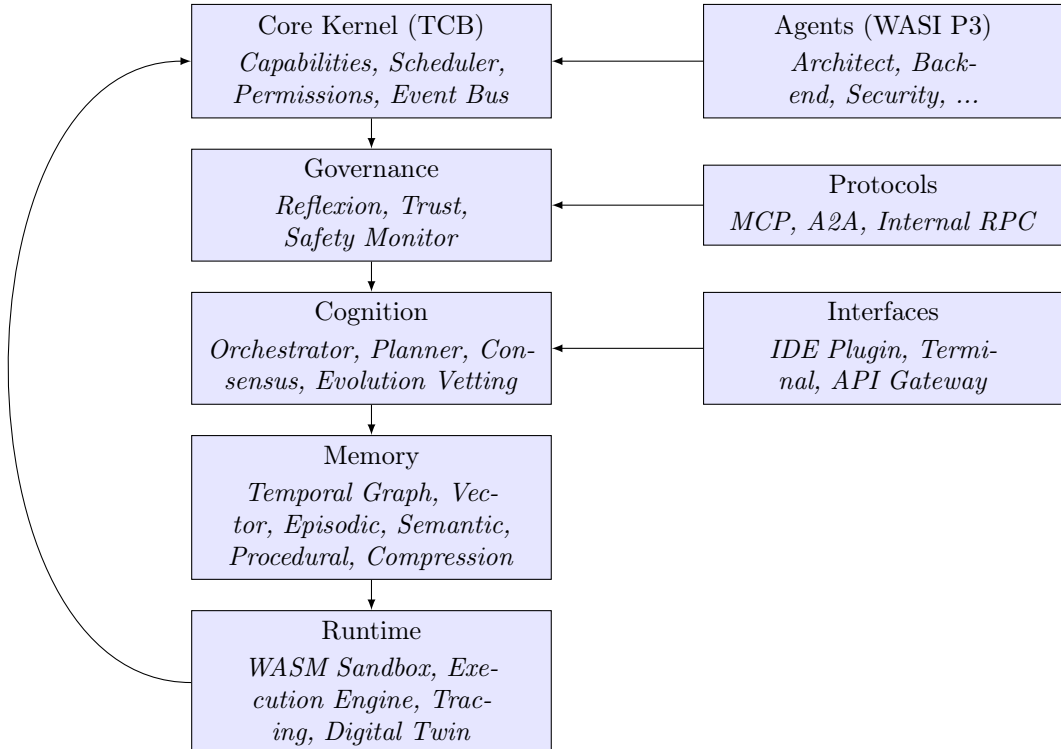
$$C_{\text{out}} = \sum_i \nu_i(\Sigma) \cdot w_i \cdot \text{opinion}_i \quad (7)$$

where w_i is a static capability weight and $\nu_i(\Sigma)$ is a dynamic trust factor derived from task success history, safety violations, and memory-coherence contributions. HACN operates in three tiers: local consensus within role groups, cross-role escalation, and global convergence under governance supervision.

3 Architecture

COS follows a **framekernel** design, inspired by the Asterinas operating system. A small, unsafe-Rust-allowed framework provides safe abstractions; all other system logic is written in safe Rust. The Trusted Computing Base (TCB) comprises only the core kernel, and it is formally verified.

3.1 Domain-of-Trust Hierarchy



The kernel core is **immutable at runtime**. Changes require an offline patch with re-verification. All inter-module communication passes through the authenticated event bus; no direct memory sharing occurs between sandboxes.

3.2 Capability Model

Every agent receives an unforgeable cryptographic token granting access to specific graph regions, sandbox resources, and communication channels. The `capability_system` verifies the token on every call according to a deny-by-default policy. An agent that steps outside its granted capabilities triggers immediate termination and increments the safety-violation counter.

3.3 WASI Preview 3 – Agent Isolation

Agents execute inside WebAssembly sandboxes using **WASI Preview 3** (WASI P3), which reached release-candidate status in late 2025. WASI P3 provides:

- First-class async/concurrency via the Component Model.
- Declarative capability granting through WIT interfaces.
- Fine-grained resource types (files, network, clocks) with deny-by-default.

Each agent is compiled to `wasm32-wasip3` and instantiated with only the capabilities explicitly granted: a virtual filesystem, standard output, and host-exposed functions. No network or system call access is permitted. Wasmtime enforces memory isolation, control-flow integrity, and type safety.

3.4 Memory: Temporal-Hybrid Graph Store

COS uses **CozoDB** as its memory substrate—an embeddable, transactional database that unifies relational, graph, and vector paradigms under a Datalog query language. Key features exploited by COS:

- **Time-travel**: soft erasure with valid-interval semantics, enabling snapshots at any past timestamp.
- **Vector search**: HNSW indices integrated with graph traversals via hybrid Datalog.
- **Provenance**: all mutations are transactional and logged for audit.

The COS schema defines `Entity` and `Relation` with temporal validity intervals, supporting post-hoc auditing and real-time memory-divergence measurement.

3.5 External Interoperability: MCP + A2A

COS adopts the two open protocols that have become the de facto standard stack for agentic AI (both under the Linux Foundation as of 2026):

- **Model Context Protocol (MCP)**: provides agent-to-tool access. COS implements a fully compliant MCP server over stdio and Streamable HTTP.
- **Agent-to-Agent (A2A) Protocol**: enables agent-to-agent coordination. COS agents expose Agent Cards and communicate via Tasks over HTTP/SSE/JSON-RPC.

The two protocols are complementary: MCP delivers tool context, A2A delivers peer delegation.

3.6 Observability and Zero-Trust

Every system call, graph mutation, and inter-agent message is recorded in a structured, cryptographically signed audit log. Telemetry streams to the governance layer, where anomaly detectors (KL spikes, unexpected capability requests) raise alerts. The digital twin can replay past states from the audit log for forensic analysis.

4 Formal Verification Strategy

Safety-critical components are verified using a layered methodology, matching technique to criticality:

Table 1: Verification Layers

Level	Target	Tool
L0	Panic-freedom, integer overflows of all <code>core/</code>	Kani model checker
L1	Functional correctness of capability system, permissions, state machine	Verus (deductive)
L2	Concurrency correctness of scheduler, event bus, lock primitives	Converos
L3	Cross-module invariants, memory-divergence bounds	Verus + KVerus (AI-assisted)

Verus is an SMT-based deductive verifier for Rust that supports proof-carrying code. **Kani** performs bounded model checking for panic-freedom and arithmetic safety. **Converos** extends Kani with concurrency-aware reduction to sequential checking, enabling practical verification of OS-level concurrent modules (used successfully on Asterinas). **KVerus** uses retrieval-augmented generation to automatically synthesise and repair Verus proofs, achieving $> 80\%$ single-file verification rates on real OS code.

The `verification_agent` will employ KVerus to auto-generate proofs for proposed evolution patches before they enter the SafeManifold, ensuring that self-evolution is both bounded and machine-checked.

5 Technology Stack

Table 2: Core Technology Choices

Layer	Technology	Rationale
Kernel	Rust (stable 2024)	Memory safety, framekernel minimizes unsafe surface
Sandbox	Wasmtime + WASI P3	Production-grade, capability-based isolation
Memory	CozoDB (embedded)	Datalog for hybrid relational/graph/vector with time-travel
Agent Inference	Burn / Candle (Rust)	100% safe inference with optional WASM backend; Python only for offline training
Protocols	MCP, A2A	Linux Foundation standards; production SDKs available
Frontend IDE	Tauri (Rust + TS)	Capability-aware, low-privilege agent execution
Observability	<code>tracing crate</code>	Structured, append-only, forensic-ready log

6 Safety and Threat Model

- **Prompt injection / agent code execution:** WASI P3 sandbox with deny-by-default capabilities; no network or host FS access. Agents are pre-compiled and immutable.
- **Capability escalation:** Cryptographic tokens verified by a Kani-proven pure function on every call.

- **Memory corruption / poisoning:** All mutations require capability tokens; audit log preserves provenance; governance monitors memory divergence and can trigger rollback.
- **Agent collusion:** Agents operate in disjoint capability domains; cross-communication requires explicit grants, logged and monitored.
- **Self-evolution runaway:** $\|\Delta\theta\|$ bounded; digital twin testing; safety regression suites; human approval before fusion.
- **Formal proof brittleness:** KVerus auto-repairs proofs after code/toolchain evolution.

Automated incident response: (1) invalid capability token $\rightarrow$ sandbox terminated, violation counter incremented; (2) memory divergence spike $\rightarrow$ automatic rollback, provenance audit; (3) safety violation threshold crossed $\rightarrow$ containment, capability revocation, human alert.

7 Related Work

Our design synthesises ideas from several established systems:

- **Asterinas** [1]: a framekernel OS in Rust with a formally verified TCB. COS adopts the framekernel pattern and its verification methodology (Verus, Converos).
- **Redox OS** [2]: a capability-based microkernel in Rust. COS follows its capability-passing model.
- **Wasmtime/WASI** [3]: provides the sandboxing substrate. COS leverages WASI P3 for fine-grained, async-native isolation.
- **CozoDB** [4]: a hybrid relational-graph-vector database with Datalog and time-travel. COS uses it as the cognitive memory store.
- **MCP and A2A** [5, 6]: open protocols for tool access and agent interoperability.
- **Verus, Kani, Converos, KVerus** [7, 8, 9, 10]: the verification toolchain enabling machine-checked safety.
- **LightThinker++ & Semantic Compressor** [11]: inspire the memory compression function Ψ .

8 Conclusion

This blueprint presents COS, a formally grounded Cognitive Operating System that rethinks agentic AI from the OS layer up. By integrating capability-based security, WASM sandboxing, temporal-hybrid graph memory, and layered formal verification, COS provides a principled foundation for persistent, safe, and continuously evolving multi-agent systems. The immediate next steps are publication of this blueprint as a research paper and the construction of the Phase 0 MVP to validate the trust architecture.

Acknowledgements. We thank the developers of Asterinas, Verus, CozoDB, and Wasmtime for their pioneering work.

References

- [1] Asterinas Project. *A safe, fast, and verified general-purpose OS kernel*. <https://github.com/asterinas/asterinas>
- [2] Redox OS. *A Unix-like Operating System written in Rust*. <https://www.redox-os.org/>
- [3] Bytecode Alliance. *Wasmtime: A fast and secure runtime for WebAssembly*. <https://wasmtime.dev/>

- [4] CozoDB. *A transactional relational-graph-vector database with Datalog*. <https://github.com/cozodb/cozo>
- [5] Anthropic. *Model Context Protocol (MCP)*. Open standard, Linux Foundation. <https://modelcontextprotocol.io/>
- [6] Google. *Agent-to-Agent (A2A) Protocol*. Open standard, Linux Foundation. <https://developers.google.com/agent-to-agent>
- [7] Verus: *Verified Rust*. <https://github.com/verus-lang/verus>
- [8] Kani Rust Verifier. <https://github.com/model-checking/kani>
- [9] Conteros. *Practical Concurrency Verification for Rust OS Modules*. 2025. (preprint)
- [10] KVerus. *AI-Assisted Proof Generation for Rust Verification*. 2026. (preprint)
- [11] LightThinker++. *Memory Compression for Long-Context Agents*. 2025. (arXiv preprint)